

can be used to do various tasks. Here jarvis resembles the bot object. This object contains various utility method like,

- hear – will listen to all messages sent and check whether it matches before firing the callback
- respond – will only analyze the message if only it is directly addressed to jarvis. i.e. starts with your hubot's name "jarvis what's the weather of Kolkata".

The callback function will have a message object and using that, the user can do one of the following three things:

- send – Sends a message to the chat
- reply – replies to the user directly
- emote – those familiar with IRC will know this to be equivalent to /me – kind of like an inline status message, and will work only where such a message is supported.

Hear and Respond

Since this is a chat bot, the most common interactions are based on messages. Hubot can hear messages said in a room or respond to messages directly addressed at it. Both methods take a regular expression and a callback function as parameters. For example:

```
alfred
jarvis.hear("hello", function(jar_resp){
  jar_resp.send("Hello
.concat(process.env.USER));
});
```

This will indeed be a basic hi-hello bot, where in hearing hello the bot will respond with the message Hello {{user_name}}.

```
jarvis.respond(/make me
coffee/i, function(jar_resp){
  jar_resp.send("Making
you coffee");
});
```

The robot.respond /make me coffee/i callback is only called for messages that are immediately preceded by the robot's name or alias. If the robot's name is JARVIS and alias is /, then this callback would be triggered for:

- jarvis make me coffee
- JARVIS: make me coffee
- @JARVIS make me coffee
- /make me coffee

It wouldn't be called for:

- Jarvis please make me coffee.

Send & reply

The jar_message parameter is an instance of Response (historically, this parameter was msg and you may see other scripts use it

*pointers

>>Interesting Hubot developer videos



*Hubot in DevOps

>>A GitHub talk on how Hubot can bring GitHub and many core DevOps tools and philosophies to developers' fingertips.

<http://dgit.in/HubDevOps>



*Hubot on Windows

>>A short tutorial on using the Hubot PowerShell Resource to install Hubot on a remote machine.

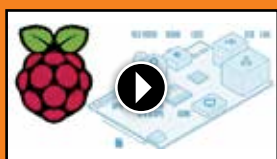
<http://dgit.in/HubPwShl>



*Custom Hubot In Slack

>> Create your own custom hubot that you can chat with in Slack that integrates with AWS.

<http://dgit.in/HubSlack>



*Raspberry Pi Chat Server with Hubot

>> Host an Internet chat server on the Raspberry Pi 2. In addition, Hubot will be configured to interact with chat room users.

<http://dgit.in/HubRbPi>



this way). With it, you can send a message back to the room from where the jar_message came from, emote a message to a room (If the given adapter supports it),

or reply to the person that sent the message.

For example:

```
module.exports = function(jarvis){
  jarvis.hear(/badger/i,
function(jar_resp){
  jar_resp.
send("Badgers? BADGERS? WE DON'T
NEED NO STINKIN BADGERS");
});
  jarvis.respond(/make me
coffee/i, function(jar_resp){
  jar_resp.reply('Yes ' +
process.env.USER + ' making you coffee');
});
  jarvis.hear(/I like
pie/i, function(jar_resp){
  jar_resp.
emote('makes a freshly baked pie');
});
}
```

The robot.hear /badgers/ callback sends a message exactly as specified regardless of who said it, "Badgers? BADGERS? WE DON'T NEED NO STINKIN BADGERS".

If a user Dave says "HAL: make me coffee", robot.respond /make me coffee/i callback sends a message "Yes Agnibha making you coffee."

Capturing data

So far, our scripts have had static responses, which, while amusing, are boring functionality-wise. res.match has the result of matching the incoming message against the regular expression. This is just a JavaScript thing, which ends up being an array with index 0 being the full text matching the expression. If you include capture groups, those will be populated res.match. For example, if we update a script like:

```
jarvis.respond(/make me (.)+/i,
function(jar_resp){
  drinkType = jar_resp.match(1);
  switch(drinkType){
    case "coffee":
      jar_resp.
send("Yes Making you brazilian Coffee");
      break;
    case "tea":
      jar_resp.
send("Making you darjeeling tea");
      break;}});
```